

Proposed Architectures A1–A9 for Sinhala/Tamil Speaker Verification

An isolated experiment pipeline, and the first results it produced

SL-SPV project

2026-08-23

Contents

1. What this is, and what it is not	2
2. The pipeline	3
2.1 The constraint that shaped it	3
2.2 How a proposal reuses the v1 recipe	3
2.3 What the trainer’s fixed interface forced	3
2.4 A constraint discovered, not designed	3
3. Results	5
3.1 A3 — metadata-conditioned calibration: a clean negative, and a large positive	5
3.2 A6 — channel-aware positive sampling: the premise does not hold here	5
3.3 A9 — reliability-driven sub-centre margins: the first positive result	6
3.4 Everything else	6
4. How this serves speaker verification for Sri Lankan languages	7
4.1 It converts “promising” into “measured”	7
4.2 It has already redirected effort	7
4.3 It makes the negative results publishable	7
4.4 It is reusable beyond these nine	7
5. Honest limitations	8
6. Reproduction	9

1. What this is, and what it is not

Benchmark v1 (report of 2026-08-22) answered *which existing system is best* for Sinhala and Tamil: a frozen multilingual SSL encoder read through learned layer weights, at 2.369 % / 0.422 % held-out test EER. The nine proposals in **proposals/** ask a different question — *what should be built next* — by implementing nine designs drawn from a 32-paper literature study and testing each against the v1 baseline it is meant to improve.

This report covers the pipeline built to run them and the results obtained so far. It is explicit throughout about which numbers exist and which do not, because the failure mode this project has repeatedly paid for is a plausible number produced by a system that was not measuring what it claimed.

Status at the time of writing: three proposals have complete results (A3, A6, A9), the rest are training or queued. Nothing below is a performance claim for an unevaluated method.

2. The pipeline

2.1 The constraint that shaped it

Every proposal had to run **without modifying voxceleb_trainer/**. That is not tidiness: `experiments/results/` holds 54 published v1 numbers, and a proposal evaluated against a modified trainer is not comparable with any of them.

The constraint is enforced, not asserted. `selftest.py` runs 13 checks including `git status --porcelain voxceleb_trainer`, and reports **13 passed, 0 failed**.

2.2 How a proposal reuses the v1 recipe

`trainSpeakerNet.py` resolves architectures and losses dynamically:

```
importlib.import_module("models." + args.model).MainModel
importlib.import_module("loss." + args.trainfunc).LossFunction
```

So a proposal supplying either is already shaped like a trainer plug-in. Two pieces close the gap:

- `_trainer_shim/` provides `models` and `loss` packages whose `__path__` is extended with the trainer’s own directories. `models.PLLW` resolves to the proposal, `models.ECAPA_TDNN` to the trainer, side by side, with nothing written inside `voxceleb_trainer/`.
- `common/train.py` builds each run by copying the **literal argv** of the matched v1 baseline and overriding only what the proposal changes. Splits, augmentation, sampler, schedule, batch size and checkpoint selection are therefore identical to the baseline, and the proposal is the only changed factor.

Copying argv rather than reconstructing from the recorded manifest is deliberate. `resolved_parameters` omits every argument the trainer passes straight through to the model, and reconstructing from it is precisely what corrupted nine v1 evaluations (manuscript §V-J, items 10–11).

2.3 What the trainer’s fixed interface forced

`trainSpeakerNet.py` exits with `unrecognized arguments` on any flag it does not declare, so a proposal cannot introduce command-line options. Three consequences, each resolved inside the proposal tree:

need	resolution
A7 names its encoder argument <code>ssl_model</code> ; the trainer passes <code>ssl_encoder_name</code>	translated in the shim adapter, so A7 and its baseline read the <i>same</i> local encoder weights
A1 needs <code>lora_rank</code> ; A8 needs <code>adv_weight</code>	passed through the environment, read by the shim adapters, recorded in each run’s provenance JSON
A1 and A8 need language/corpus labels; the batch carries only a speaker index	<code>common/domains.py</code> derives them from the same <code>train_list.txt</code> the run trains on

Deriving the domain labels rather than annotating them matters: a side-car label file is a second source of truth that can drift from the split it describes, and drift of exactly that kind is what this project has lost the most time to.

2.4 A constraint discovered, not designed

`DomainTable` reports that the `si` and `ta` conditions each contain **one language and one corpus**. A1, A6, A7 and A8 all depend on a domain variable: A1, A6 and A8 are adversarial or disentangling over one, and A7 assigns a layer-weight vector per language. With one class, every discriminator is constant, every gradient-reversal term is uninformative, the per-language vectors collapse to one, and each method silently degrades to its own baseline **while still producing a number**.

The adapters therefore **refuse** that case rather than reporting it, and those three proposals target **combined_si_ta** (782 speakers; si 336 / ta 446; two corpora) or **si_pooled**. v1 never trained **combined**, so the matched baseline had to be added — **train_baseline.py** — and is itself the programme’s first multi-language system.

This is the most consequential thing the pipeline surfaced. **Four of nine proposals are undefined on the conditions the benchmark was built from**, and nothing in their papers or READMEs says so.

A7 is the cautionary case, because the guard was added only after it had already run. Trained on **ta** (one language) with **n_languages** left at its default of 3, it built three weight vectors and a three-way LID head on data containing one language, trained to completion, and scored **0.663 % test EER against its shared-weight baseline’s 0.623 %**. That reads as a small regression by a plausible margin. It is not a regression: it is shared layer weighting plus two dead vectors and a degenerate head, and it measures nothing about per-language weighting. The run has been retired to **A7_pllw/out/_degenerate_single_language/** with that explanation, A7 now refuses a single-language condition, and it is requeued on **combined**.

The lesson is the one this programme keeps relearning: **a method that is undefined on the data still trains, still converges, and still emits a number in the right range**. Only a guard derived from the data catches it.

3. Results

3.1 A3 — metadata-conditioned calibration: a clean negative, and a large positive

A3 fits a training-free calibrator on per-trial scores, conditioning on trial metadata (language, corpus, evaluation set). Run across **33 v1 systems** with four controls per system.

	median across 33 systems
Cllr, raw cosine	0.8356
Cllr, after calibration	0.1873
EER significantly better than the matched score-only control	0 / 33
systems where <i>shuffling</i> the metadata changes EER by $< 10^{-9}$	29 / 33

Two findings, and they point in opposite directions:

Finding P1 — calibration is a large, free win. Cllr falls by a median factor of **4.5x** at 1,225 parameters, no GPU and no retraining. For a deployed verifier this is the difference between scores that carry a usable likelihood-ratio interpretation and scores that do not. It is the cheapest improvement measured anywhere in this programme.

Finding P2 — the metadata contributes nothing. The gain is entirely score-domain. A matched score-only control reaches the same EER on every system, no contrast is significant under the paired speaker-clustered bootstrap, and in 29 of 33 systems **replacing the metadata with a shuffled copy changes the EER by less than 10-9**. The metadata-only negative control sits at 98–99 % EER, confirming the harness measures what it claims.

The shuffled-metadata control is what makes P2 strong rather than merely underpowered. A null from the bootstrap alone would be ambiguous at 18–26 test speakers; a *bit-identical* result under shuffling is not ambiguous. The metadata is inert.

What P2 does not say. It does not say metadata conditioning cannot work. It says that on these corpora, with language/corpus/set as the conditioning variables, it adds nothing over calibrating the scores. A collection with genuinely heterogeneous enrolment conditions might differ.

3.2 A6 — channel-aware positive sampling: the premise does not hold here

A6 argues that SSPS harvests same-*channel* rather than same-*speaker* pairs, and corrects it by projecting the channel subspace out before clustering. Both arms run the identical pipeline on **si_pooled** and differ only in that projection.

arm	speaker purity	speaker NMI	corpus purity	corpus NMI
plain SSPS (projection off)	0.9302	0.9810	0.9993	0.2859
channel-aware (projection on)	0.9099	0.9767	0.9922	0.2779

Finding P3 — the correction is unnecessary here, and slightly harmful. Clusters are already speaker-shaped: speaker NMI 0.98 against corpus NMI 0.28. Removing eight channel directions *reduces* speaker purity by 2.0 pp. Both arms’ own diagnostic returns the same reading — “*clusters are speaker-shaped; SSPS as published may suffice*”.

This does not contradict v1’s channel finding, and the distinction is worth stating precisely. v1 showed channel is **linearly decodable** from the embedding (corpus predicted at 91.2 % against 20 % chance). A6’s premise is stronger: that channel dominates the **local neighbourhood structure**. The first can hold while the second fails, and here it does.

A limitation that materially bounds this result. The bootstrap embeddings come from A_escapa1024_si_pooled, a **supervised** speaker model trained on these speakers. Its space is speaker-organised

by construction, which is exactly the condition under which A6’s premise should fail. Genuine SSPS bootstraps from a self-supervised model with no speaker labels. **P3 is therefore evidence about this setup, not about SSPS in its intended setting**, and the honest next step is to repeat it from a self-supervised bootstrap.

3.3 A9 — reliability-driven sub-centre margins: the first positive result

A9 replaces AAM-Softmax with a sub-centre variant in which K is set **per corpus from a measured label-reliability statistic** rather than fixed at the customary $K = 3$. The loss is the only changed factor: same recipe, same splits, same backbone, same schedule.

language	baseline (AAM)	A9 (sub-centre)	Delta	95 % CI	p	significant
Sinhala	4.375	3.962	-0.413 pp	[-0.677, -0.015]	0.038	yes
Tamil	0.884	0.934	+0.050 pp	[-0.109, +0.195]	0.668	no

Paired speaker-clustered bootstrap, 2,000 resamples, identical trials (91 / 131 speaker clusters). AS-Norm and minDCF move the same way (si 3.912 $\rightarrow$ 3.781; 0.3100 $\rightarrow$ 0.2817).

Finding P4 — sub-centres help on Sinhala and not on Tamil, and the split follows the audit.

This is the asymmetry the QC audit predicts. SLR127 (Tamil) is the corpus whose speaker-identity error the audit *found and corrected*: 638 speakers had been collapsed into 531 labels, and the v1 splits use the corrected key. SLR52 (Sinhala) is the corpus the audit flagged as unresolved — its label-reliability profile is consistent with account-level rather than person-level identity, and no metric can settle that without a listening pass.

Sub-centres are the standard response to residual within-class identity heterogeneity. They improve the corpus where such heterogeneity is *suspected but uncorrected*, and do nothing measurable on the corpus where it was *found and fixed*. That is a mechanistically coherent result rather than a bare number, and it is the first evidence in this programme that a proposal earns its place.

How strongly to hold it. $p = 0.038$ with an interval reaching -0.015 pp is marginal, and this is a single seed. The result is worth a three-seed replication before it carries weight in a paper, and it is stated here as promising rather than settled.

3.4 Everything else

proposal	state
A2 CC-NAP	2 systems scored; requires ≥ 2 languages in the fitting pool
A7 PLLW	first runs retired as degenerate (single-language, §2.4); requeued on combined
A1 DA2-LoRA	verified end to end (287.9 M total, 4.19 M trainable, 1.46 %); queued at ~ 44 min/epoch
A4 LS-CAM, A5 DK-CAM++	training
A8 CD-NDAL	verified end to end on combined ; queued
baseline combined	training — the control A1 and A8 need

No EER is reported for any of these, because none has been scored yet.

4. How this serves speaker verification for Sri Lankan languages

4.1 It converts “promising” into “measured”

The literature study produced nine plausible designs. Plausibility is cheap; each README argues convincingly for its method. The pipeline’s contribution is that a proposal now has to survive a **matched, single-factor comparison against a published v1 number** before it can be called an improvement. Three have now been tested that way: **two came back negative and one positive** — which is exactly what a pipeline is for, and exactly what would not have been reached by reading the papers, all three of which argue persuasively.

4.2 It has already redirected effort

- **Calibrate, do not annotate.** P1 and P2 together say that a Sri Lankan deployment should invest in score calibration — 4.5x better Cllr for 1,225 parameters — and should *not* invest in the metadata plumbing that A3’s full form would require. That is a concrete, immediate saving.
- **The channel confound is real but shallow.** P3 sharpens v1’s channel finding: channel is linearly decodable but does not dominate local neighbourhoods. Work aimed at the confound should target the *scoring* stage, where v1 measured it (48 % of a published Tamil impostor list rejected on channel), rather than the clustering stage.
- **Four methods need data the benchmark does not have.** A1, A6, A7 and A8 all need a domain variable. The collection supplies one only by pooling corpora, which confounds domain with language or with recording era. **Acquiring within-speaker, cross-channel Sri Lankan recordings would unlock four of nine proposals at once** — a clearer data priority than any of the nine methods.

4.3 It makes the negative results publishable

Under-resourced-language speaker verification has very little published groundwork, and almost none of it negative. A controlled null — *metadata conditioning adds nothing over score calibration on Sinhala and Tamil*, with a shuffled-metadata control demonstrating inertness rather than merely absent significance — is a contribution precisely because it stops the next group spending months on the same idea.

4.4 It is reusable beyond these nine

`_trainer_shim + common/train.py + common/domains.py + evaluate_proposal.py` form a general harness: any future method that supplies a `MainModel` or a `LossFunction` can be trained on the v1 recipe and scored on the v1 trials with one changed factor, without touching the benchmark. That is the piece of this work most likely to outlive the specific proposals.

5. Honest limitations

1. **Three of nine have complete results.** The rest are queued or in training; no claim is made for them. A7’s first result was withdrawn after the run was found degenerate (§2.4), which is recorded rather than quietly removed.
2. **Single seed** throughout, as in v1. At 18–26 test speakers for A3’s speaker-split protocol, absolute EER carries a wide interval; only paired contrasts and the shuffled-metadata control support the conclusions drawn.
3. **A6’s bootstrap is supervised**, which biases its result toward the negative finding (§3.2).
4. **A8 drops the reconstruction regulariser** because the trainer supplies one view per sample. Only its K-way corpus-adversarial claim is tested, and the deviation is recorded in each run’s output.
5. **combined confounds language with corpus** — its two corpora are the Sinhala and Tamil sources — so an adversary trained there cannot separate “corpus” from “language”. **si_pooled** is the cleaner domain contrast and is the better home for A8.

6. Reproduction

```
cd proposals
python3 selftest.py           # 13 checks, CPU, < 1 min
python3 scaffold.py --check   # which proposals have real results
python3 run_all.py --status   # training queue
python3 evaluate_proposal.py --list # which trained runs are scored
python3 proposal_queue.py --plan # score-level proposals across CPU/GPU
```

`scaffold.py` counts a proposal as having results only when `out/` holds a JSON from a **successful** run — neither `*.command.json` provenance nor a `status: FAILED` record counts. It reported 9/9 on 2026-08-23 while the queue correctly reported 2/11; that discrepancy was the bug, and it is fixed.